Not Only SQL

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

TASK 2 NOSQL (40%)

Scenario:

A small e-commerce store needs to store data about its customers, their purchases, and the points they accumulate. Customers earn points based on the value of their purchases, and the store wants to track this information to offer discounts and rewards. The store needs a database to efficiently store and retrieve customer and purchase data.

You are required to design a simple NoSQL database to handle customer data, their purchases, and points.

Question 1:

Explain the concept of NoSQL databases and describe how it would be useful for storing customer and purchase data for an e-commerce store. Compare it briefly with traditional relational databases.

Answer:

NoSQL Databases, which stands for "**Not Only SQL**", are different from tradition databases because they don't use tables in the normal way. Instead, they store data in flexible formats like documents or key-value pairs, which makes them easier to work with when data isn't always what you want.

Talking about **E-Commerce System**, not every customer that you attend have same info. Some might give you the full detailed information but some might give you basics one. With the help of NoSQL, you don't need to worry about empty fields in the collection because each record can store only what's needed.

Its also useful because e-commerce systems deal with different types of data like orders, reviews, and user activity. Since the data keeps changing, NoSQL makes it easier to manage without having to restructure every data again and again.

Another advantage is that it can handle high rush traffics. During sales or busy hour, more users come in, and NoSQL databases can manage this by scaling easily keeping the system running smoothly.

Comparison: NoSQL vs Relational Databases

Feature	NoSQL Databases	Relational Databases
Structure	Data is stored in flexible formats like. json, key-value, etc.	Data is stored in tables with rows and columns
Schema	No fixed schema, can change easily	Fixed schema, planning is done beforehand
Data Type	Can handle structured, semi-structed, and unstructured data	Mainly works with structured data
Scalability	Easy to scale by adding more servers	Scaling is hard, done by upgrading one system
Performance	Faster for large and frequently changing data	Can slow down when data grows and joins are used
Joins	Doesn't rely on joins, data is stored together	Depends upon multiple joins to connect tables
Flexibility	Very flexible	Less flexible
Use Case	Used in apps like e-commerce, social media, etc.	Used in systems like banking, ERP
Consistency	Eventual Consistency	Strongly consistent with ACID Properties

Question 2:

Assume you are using MongoDB (document-based NoSQL) to store customer and purchase data. Write the MongoDB script to:

1. Create a collection for customers.

2. Create a collection for purchases.

3. Insert sample data for a few customers and their purchases (including points).

Answer:

1. I created **Database** and **Customers Collection**:

```
>_MONGOSH

> use ecommerceDB
< switched to db ecommerceDB
ecommerceDB>
```

```
> db.createCollection("customers")
< { ok: 1 }
```

2. I created a **Purchases Collection**:

```
> db.createCollection("purchases")
< { ok: 1 }
```

3. Inserted Sample Data:

A. **Customers Collection**

```
> db.customers.insertMany([
    {
      customer_id: 101,
      name: "Ronak",
      email: "ronu@example.com",
      phone: "9812345678",
      points: 500
    },
    {
      customer_id: 102,
      name: "Bhabya",
      email: "pari@example.com",
      points: 300
    },
    {
      customer_id: 103,
      name: "Raya",
      email: "raya@example.com",
      phone: "9800000000",
      points: 700
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69f4c32225e07bab5476b4e3'),
      '1': ObjectId('69f4c32225e07bab5476b4e4'),
      '2': ObjectId('69f4c32225e07bab5476b4e5')
    }
  }
```

B. **Purchases Collection**

```
> db.purchases.insertMany([
    {
      purchase_id: 1,
      customer_id: 101,
      product: "Trimmer",
      amount: 800,
      date: "2026-04-01"
    },
    {
      purchase_id: 2,
      customer_id: 101,
      product: "T.V.",
      amount: 20,
      date: "2026-04-02"
    },
    {
      purchase_id: 3,
      customer_id: 102,
      product: "Laptop",
      amount: 50,
      date: "2026-04-03"
    },
    {
      purchase_id: 4,
      customer_id: 103,
      product: "Phone",
      amount: 600,
      date: "2026-04-04"
    }
  ])
```

```
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69f4c37225e07bab5476b4e6'),
      '1': ObjectId('69f4c37225e07bab5476b4e7'),
      '2': ObjectId('69f4c37225e07bab5476b4e8'),
      '3': ObjectId('69f4c37225e07bab5476b4e9')
    }
  }
```

For this task, a MongoDB database was created named **ecommerceDB.**

After that, at first, I created collection for **customers** and then similarly I created collection for **purchases**.

Customers were created to store customer details whereas purchases were created to record the transaction of the products sold.

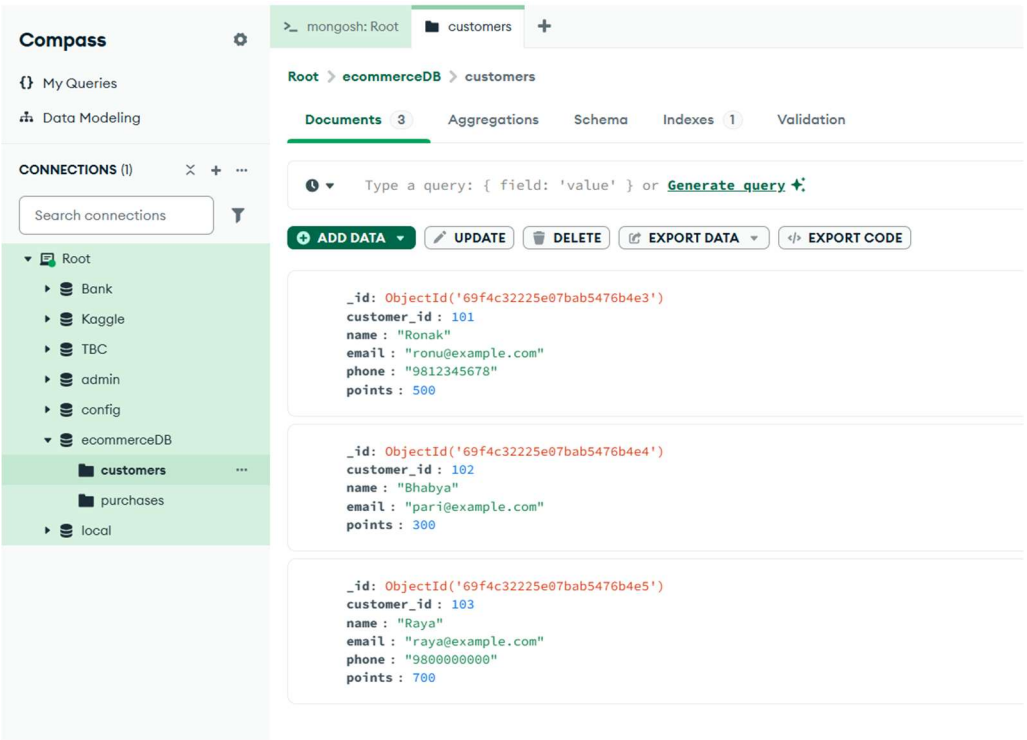
I added sample data using the **insertMany ()** command.

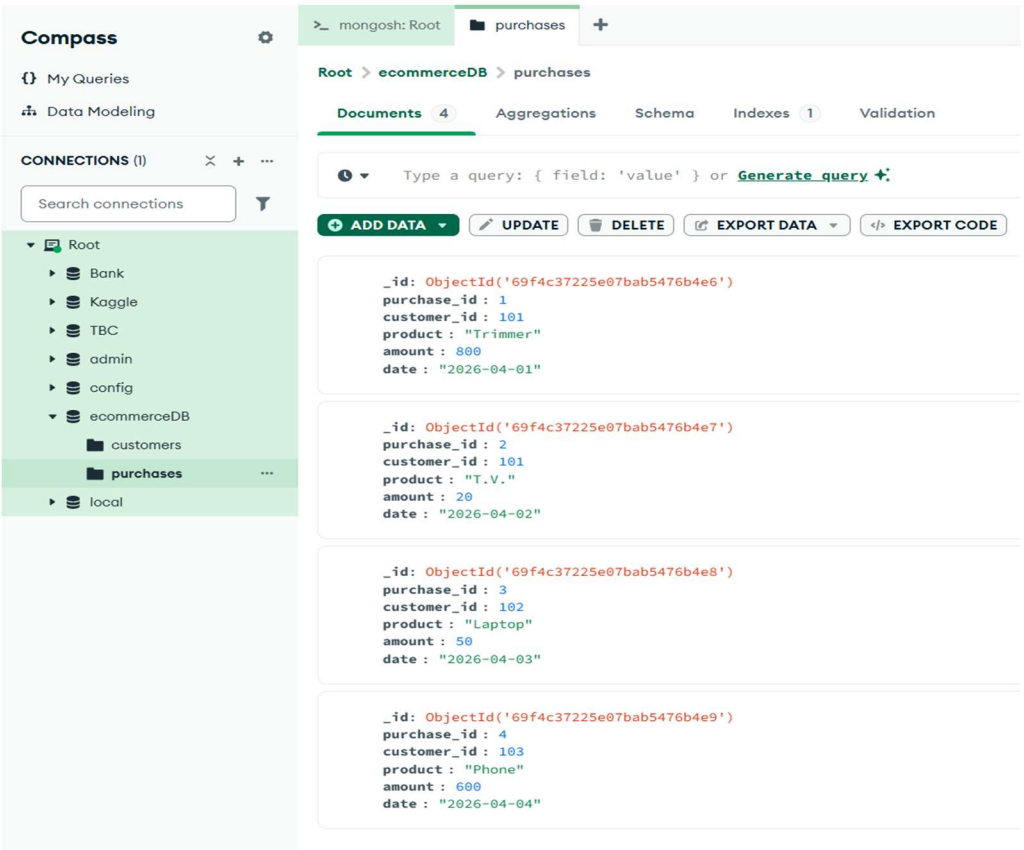
Each customer has a unique **customer_id**, which is also used in purchases collection to connect each purchase with the correct customer.

The **point** field in the **customers collection** represents reward points given if the purchase was made.

This structure keeps the data simple, flexible and easy to access and manage, which is one of the major advantages of using NoSQL database.

Visual Representation:





Question 3:

What are some potential challenges of using NoSQL databases like MongoDB for an e-commerce store, and how would you mitigate these challenges?

Answer:

I have listed out five major challenges and its respective solutions. They are as follow:

1. What if Data is Duplicated?

In MongoDB, data is often stored together to improve the performance. Because of this, the same data can appear in multiple places. If something changes, it needs to be updated everywhere, which is more time consuming and difficult also.

Solution:

We will use proper data design. We will frequently change the data and also, we will use referencing instead of repeating same data everywhere.

2.What if it's hard to maintain data consistency?

MongoDB may not always update data instantly. This can be a problem in situation like payments or inventory updates.

Solution:

We will use transactions where necessary and we will add proper validation in the application to ensure correct data handling.

3. What if we have security risks?

If MongoDB is not configured properly, it can expose sensitive data such as customer details which is very bad for companies' reputation.

Solution:

We will enable authentication, use of access control will be implemented, and we will apply encryption. We will also keep the data up to dated.

4. What if we have performance issues?

If the database is not designed properly, queries can become slow, especially with large dataset.

Solution:

We will be using indexing, we will design the schema carefully, and we will monitor our performance regularly.

Conclusion:

Although MongoDB has some challenges, it is not that we can not mitigate it. We can follow all the given solutions to overcome these challenges and run the e-commerce system smoothly.

Tools and Technology Used:

1. MongoDB

I used **MongoDB** to create a database and the run the scripts according to the questions. Mainly I used the terminal shell in the MongoDB to do the task.